SEMANA 4 · PARTE 3

Ivett Anahi Zaragocin Mora

Interrupciones externas en ESP32

BANDERA · REBOTE · TODO JUNTO

Bloque 8 · El patrón bandera

Patrón bandera: la ISR es corta: solo sube una bandera; el trabajo pesado lo hace `loop()`.

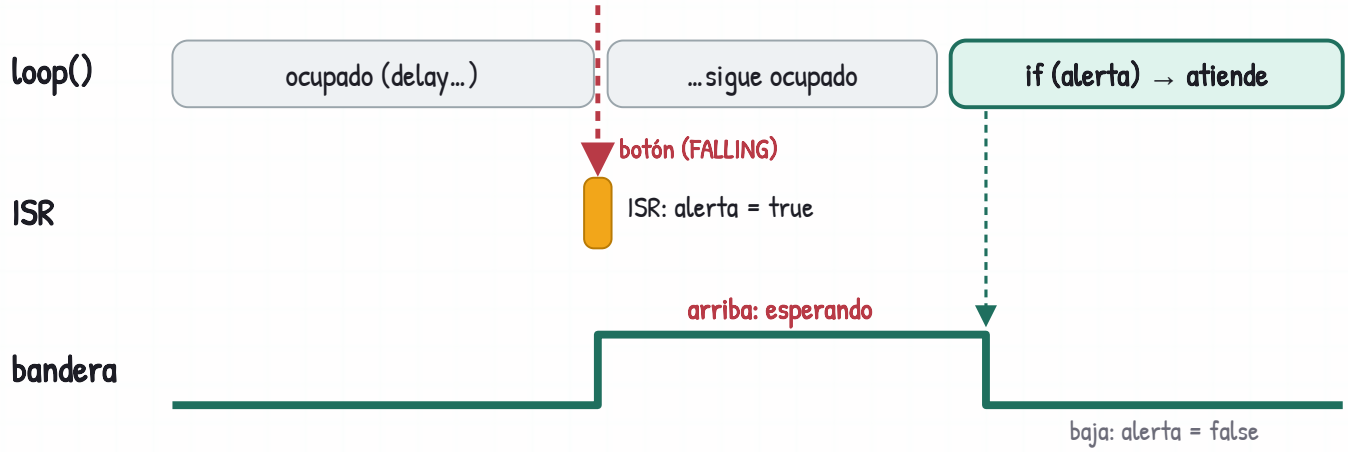
ISR (rápida)

1. Sube la bandera: `alerta = true`
2. Sale.

Nada más.

loop() (con calma)

1. ¿Bandera arriba?
2. La baja **antes** de trabajar: `alerta = false`
3. Trabajo pesado: imprimir, Wi-Fi, LCD…



ISR sube la bandera → el loop (ocupado) la ve más tarde y la baja

La bandera en código

```
volatile bool alerta = false;      // 1. la bandera

void IRAM_ATTR isrBoton() {        // 2. la ISR
  alerta = true;                   //    sube la bandera
}

void loop() {                      // 3. el loop
  if (alerta) {                    //    ¿arriba?
    alerta = false;                //    la baja
    Serial.println("¡Alerta!");    //    trabajo pesado
  }
}
```

volatile	ISR y loop comparten la variable: el compilador no debe «cachearla».
IRAM_ATTR	La ISR vive en la RAM interna (IRAM) para ejecutarse rápido y segura.
if (alerta)	Sondea la bandera; baja **antes** de trabajar.

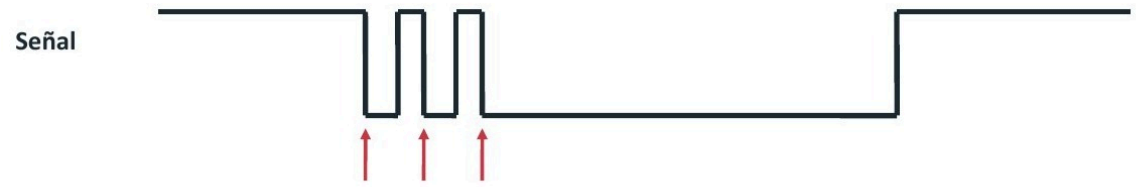
La bandera no cuenta

```
volatile uint32_t pulsos = 0;
void IRAM_ATTR isrBoton() { pulsos++; }
```

Necesito saber...	Uso	Tipo
Si ocurrió el evento (sí / no)	Bandera	`volatile bool`
Cuántas veces ocurrió	Contador	`volatile uint32_t`

Bloque 9 · El rebote

Rebote: un toque en el pulsador produce **varios flancos** y cada uno dispara la ISR.



3 flancos de bajada en unos milisegundos → la ISR corre 3 veces

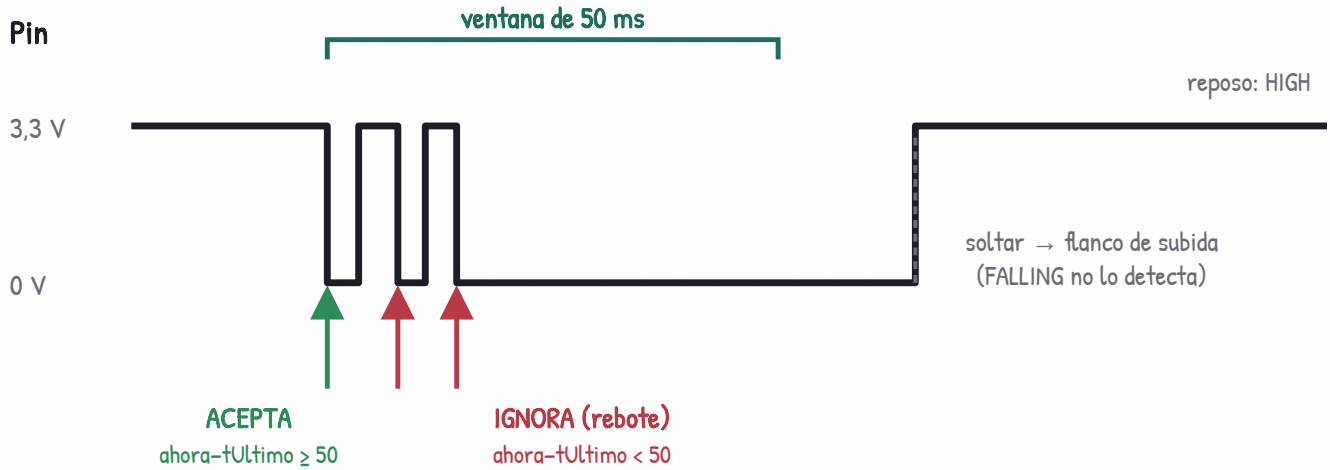
Antirrebote con millis() en la ISR

```
uint32_t tUltimo = 0;        // solo ISR

void IRAM_ATTR isrBoton() {
  uint32_t ahora = millis();
  // ¿toque nuevo?
  if (ahora - tUltimo ⩾ 50) {
    alerta = true;
    tUltimo = ahora;
  } // si no: es rebote
}
```

✦ La idea

Ignorar flancos a **menos de 50 ms** del último aceptado (patrón `ahora - tAnterior` de la semana 2).



Solo el primer flanco dentro de la ventana activa la bandera

💡 `tUltimo` solo la usa la ISR: **sin volatile**. `alerta` la comparten ISR y loop: **volatile**.

Bloque 10 · Todo junto

✦ Recorrido de un aviso

Botón FALLING → **Periférico GPIO** detecta → **ISR** IRAM_ATTR + antirrebote → **Bandera** volatile → **loop()** trabajo pesado

✦ Declaraciones e ISR

```
const uint8_t  PIN_BOTON = 21;
const uint8_t  PIN_LED   = 19;
const uint32_t T_REBOTE  = 50;          // ms

volatile bool alerta = false;           // ISR y loop
uint32_t tUltimo = 0;                   // solo la ISR

void IRAM_ATTR isrBoton() {
  uint32_t ahora = millis();
  if (ahora - tUltimo ⩾ T_REBOTE) {
    alerta = true;
    tUltimo = ahora;
  }
}
```

✦ setup() y loop()

```
void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_BOTON, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_BOTON),
                  isrBoton, FALLING);
}

void loop() {
  if (alerta) {
    alerta = false;
    digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    Serial.println("¡Alerta! Puerta abierta");
  }
  delay(2000);   // loop ocupado: el aviso NO se pierde
}
```

Control de aprendizaje

1 Flanco correcto

Un pulsador está conectado con `INPUT_PULLUP`. La alarma debe activarse en el instante en que alguien **presiona** el botón. ¿Qué modo usas en `attachInterrupt`: RISING, FALLING o CHANGE? Justifica.

> **RESPUESTA** **FALLING.** Con pull-up el pin está en HIGH en reposo; al presionar se conecta a GND y baja a LOW, es decir, un flanco de bajada. RISING dispararía al *soltar* y CHANGE dispararía dos veces (al presionar y al soltar).

2 ¿Cuáles necesitan volatile?

Tres variables globales; ¿cuáles deben declararse `volatile`? Justifica cada una.

```
uint32_t pulsos;    // la ISR suma 1; el loop lee
uint32_t tUltimo;   // solo la usa la ISR
uint32_t total;     // solo la usa el loop
```

RESPUESTA Solo `pulsos`: la modifica la ISR y la lee el loop, así que el compilador no debe asumir que no cambia. `tUltimo` la usa solo la ISR y `total` solo el loop: no se comparten entre contextos, no necesitan `volatile`.

3 El ESP32 se reinicia

Un estudiante escribe `Serial.println("toque")` dentro de la ISR. De vez en cuando el ESP32 se reinicia y el Monitor Serie muestra «Guru Meditation Error». ¿Qué está pasando y cómo lo corriges?

RESPUESTA `Serial` es lento, bloqueante y no está pensado para ejecutarse dentro de una ISR (usa funciones fuera de IRAM y puede esperar al buffer). Eso provoca el error y el reinicio. **Corrección:** la ISR solo sube la bandera `alerta = true` y el `Serial.println` se hace en `loop()`.

4 Sin antirrebote

Se quita el antirrebote de la ISR (`T_REBOTE = 0`) y se presiona el botón 10 veces. ¿Cuántos avisos esperas: exactamente 10, menos de 10 o más de 10? Justifica.

RESPUESTA **Más de 10.** Cada toque genera varios flancos de bajada por el rebote y con `T_REBOTE = 0` todos pasan el filtro; cada uno dispara la ISR y cuenta como un aviso (un toque «vale» 3 o 5).

5 Repaso de temporizadores (semana 3)

Un temporizador cuenta a 1 MHz (una cuenta cada 1 µs). La alarma está en 500 000 con autorrecarga. ¿Cada cuánto tiempo se ejecuta la ISR? ¿Cuántas veces por segundo? Muestra el cálculo.

RESPUESTA Periodo = 500 000 × 1 µs = 500 000 µs = **0,5 s** (500 ms). Frecuencia = 1 / 0,5 s = **2 veces por segundo** (2 Hz).

Resumen rápido

- ISR corta: solo `alerta = true`; `loop()` baja la bandera y trabaja. Para contar, un contador `volatile uint32_t`.
- Rebote: un toque = varios flancos; se filtra con `ahora - tUltimo ≥ 50 ms` dentro de la ISR.
- Con `INPUT_PULLUP`, presionar = `FALLING`; la ISR lleva `IRAM_ATTR` y lo compartido, `volatile`.
- Nunca `Serial`, Wi-Fi ni LCD dentro de la ISR.